

Tutor MX: Using PostgreSQL + Express Instead of Supabase

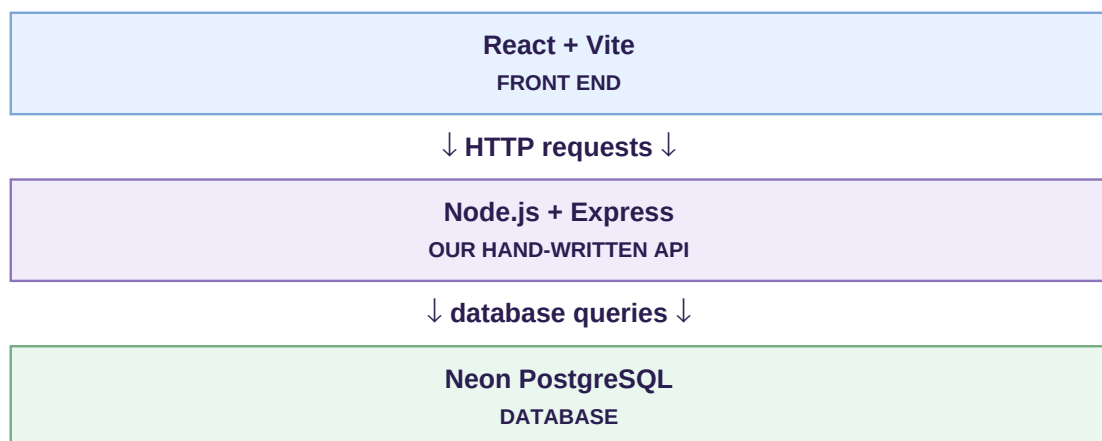
Simple team explanation based on the COMS3011A project brief

Why change? The brief says the team must design and implement its own API and warns against systems that generate API endpoints automatically. Supabase is specifically listed as an example. Using Express + PostgreSQL makes it clear that **our team wrote the API**.

1. Recommended stack

Part	Use	What it does
Front end	React + Vite	Pages, forms and dashboards.
Back end	Node.js + Express	Our hand-written API and server logic.
Database	PostgreSQL	Stores users, tutors, courses, timesheets, etc.
Database hosting	Neon PostgreSQL	Hosts PostgreSQL online.
Database access	Prisma or pg	Lets Express query PostgreSQL.
Authentication	Established auth library	Secure sign-up/sign-in using established libraries.
Front-end hosting	Cloudflare Pages	Publishes the React site.
Back-end hosting	Render or similar	Runs the Express server.
Version control / CI	GitHub + Actions	Collaboration, tests and CI/CD.
Documentation	MkDocs / Docusaurus	Public documentation website.

2. How it works



The important point is that the React app does **not** talk directly to the database. It talks to our Express API, and Express talks to PostgreSQL.

3. Why this is safer for the brief

- **Hand-written API:** our group creates routes such as GET /api/tutors and POST /api/timesheets.
- **Clear front/back separation:** React is the front end; Express is the back end.
- **Database only:** Neon hosts PostgreSQL but does not replace our business API.
- **Easy to demonstrate:** GitHub commits show who built each route, page, test and database feature.
- **CI/CD friendly:** GitHub Actions can run tests automatically on pushes and pull requests.

4. Supabase vs Express + PostgreSQL

Question	Supabase	Express + PostgreSQL
Who creates the API?	Supabase can generate database API endpoints.	Our team manually creates the Express routes.
Hand-written API requirement	Could be risky unless the tutor approves the exact usage.	Clear and easy to prove.
Front/back separation	Front end may call Supabase directly.	React -> Express -> PostgreSQL.
Difficulty	Quicker to start.	Slightly more work, but directly practises back-end/API skills.

5. Example routes our team would create

```
GET /api/tutors
POST /api/tutors
GET /api/tutors/:id
PUT /api/tutors/:id
DELETE /api/tutors/:id
```

These endpoints live inside our own Express project. That is the main difference from relying on automatically generated database endpoints.

6. Suggested GitHub structure

```
Tutor-MX/
  frontend/      # React + Vite
  backend/       # Express API + tests
  docs/          # MkDocs / Docusaurus
  .github/workflows/ # CI/CD
  README.md
```

7. Is this free?

The software itself - React, Vite, Node.js, Express, PostgreSQL, Prisma/pg, Git and common testing/documentation tools - is free and open source. Neon, Cloudflare Pages and some back-end hosting providers have free tiers suitable for student projects. Free-tier limits can change, so the team should check them before final deployment.

8. Requirements we still need to complete

- **Authentication and security:** sign-up, sign-in, password reset and account deletion using an established authentication library/service.
- **External API integration:** the project still needs one relevant external API service. This is separate from our own Express API.
- **Documentation website:** it must be publicly available.
- **CI/CD:** the team still needs automated collaboration/deployment processes.
- **Responsive and accessible front end:** the React site must meet the brief's web standards.

Recommended setup:

React + Vite (front end) -> **Node.js + Express** (our API) -> **PostgreSQL on Neon** (database).

This avoids relying on Supabase-generated API endpoints and makes the project's hand-written API requirement much easier to demonstrate.

Before the team commits to the final stack, confirm the authentication choice and hosted services with the tutor.